

Integrating Qdrant with Hugging Face LLMs for RAG

Technical Guide

September 2025

1 Introduction

Large Language Models (LLMs) excel at generating human-like text but often require external context to provide accurate, grounded responses. Retrieval-Augmented Generation (RAG) addresses this by combining LLMs with a retrieval system. This document outlines how to integrate Qdrant, an open-source vector database, with Hugging Face's LLMs to build an efficient RAG pipeline. Qdrant enables fast similarity search for embeddings, while Hugging Face provides pre-trained models and embedding tools.

Key benefits:

- Scalable storage of high-dimensional vectors.
- Real-time semantic search for relevant context.
- Easy integration via Python APIs.

2 Overview

2.1 Large Language Models

Hugging Face's `transformers` library offers access to LLMs like Llama or GPT-based models. These models generate text but benefit from retrieved context to reduce inaccuracies.

2.2 Qdrant Vector Database

Qdrant stores and searches vectors using HNSW indexing for approximate nearest neighbor (ANN) search. It supports metadata filtering and is optimized for semantic search.

3 Setup

3.1 Installation

Install required libraries:

```
pip install qdrant-client transformers sentence-transformers torch
```

Run Qdrant via Docker:

```
docker run -p 6333:6333 -v $(pwd)/qdrant_storage:/qdrant/storage qdrant/qdrant
```

Connect in Python:

```
from qdrant_client import QdrantClient
```

```
client = QdrantClient("localhost", port=6333)
```

4 Generating and Storing Embeddings

Use Sentence Transformers to create embeddings:

```
from sentence_transformers import SentenceTransformer
```

```
embedder = SentenceTransformer('all-MiniLM-L6-v2')
```

```
documents = ["AI is transforming industries.", "Qdrant powers vector search."]
```

```
embeddings = embedder.encode(documents)
```

Store in Qdrant:

```
from qdrant_client.http import models
```

```
client.create_collection(
    collection_name="docs",
    vectors_config=models.VectorParams(size=384, distance=models.Distance.COSINE)
)
client.upsert(
    collection_name="docs",
    points=[
        models.PointStruct(id=idx, vector=emb.tolist(), payload={"text": doc})
        for idx, (emb, doc) in enumerate(zip(embeddings, documents))
    ]
)
```

5 Retrieval and Generation

Search for relevant documents:

```
query = "AI advancements"
```

```
query_embedding = embedder.encode([query])[0]
```

```
hits = client.search(collection_name="docs", query_vector=query_embedding.tolist(), limit=10)
```

```
context = [hit.payload['text'] for hit in hits]
```

Generate a response using an LLM:

```
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```
model_name = "gpt2" % Replace with desired model
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(model_name)
```

```
prompt = f"Context: {' '.join(context)}\nQuestion: What is AI's impact?\nAnswer:"
```

```
inputs = tokenizer(prompt, return_tensors="pt")
```

```
outputs = model.generate(*inputs, max_length=100)
```

```
response = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
print(response)
```

6 Best Practices

- Batch embedding generation for efficiency.
- Use filters in Qdrant for targeted retrieval.
- Monitor Qdrant performance with Prometheus.

7 Conclusion

Combining Qdrant with Hugging Face LLMs creates a powerful RAG system for applications like knowledge bases or chatbots. For more details, see [Qdrant Docs](#) and [Hugging Face Docs](#).